

How to Get Started

This is a short walk through the `pyrulearn` package for inductive rule learning: load your own data from a CSV, fit a native rule learner, inspect the rules it found in a few different formats, and compare two learners across a handful of the benchmark datasets. See the [README](#) for the fuller picture – every learner, interface and demo.

1 Load data from a CSV

`titanic.csv` next to this notebook is the classic Titanic dataset – one row per passenger, `survived` the target. A plain look at it first, the way you'd look at any CSV:

```
[1]: import pandas as pd

df = pd.read_csv("titanic.csv")
df.head()
```

```
[1]:  pclass  survived    sex    age  sibsp  parch    fare embarked
0      1  survived  female  29.0000    0     0  211.3375         S
1      1  survived   male   0.9167    1     2  151.5500         S
2      1    died  female   2.0000    1     2  151.5500         S
3      1    died   male  30.0000    1     2  151.5500         S
4      1    died  female  25.0000    1     2  151.5500         S
```

`pyrulearn.data.io.read_csv` reads the same file straight into a `BooleanDataRepresentation`: it guesses each column's type (numeric vs. nominal), discretizes the numeric ones, and expands everything into binary features – each with its explicit negation – ready to fit on.

Numeric columns are split into `max_intervals` intervals (supervised, decision-tree-based thresholds): more intervals give finer-grained thresholds (more binary features, slower, more prone to overfitting on small data), fewer give coarser ones. The default is 8; we use 3 here.

```
[2]: from pyrulearn.data.io import read_csv

data = read_csv("titanic.csv", target="survived", max_intervals=3)
print(data.spec.n_features,
      "binary features from",
      len(df.columns) - 1,
      "raw columns")
```

52 binary features from 7 raw columns

2 Internal representation

`read_csv` (and every other loader) returns a `BooleanDataRepresentation`: `data.X` is the actual 0/1 matrix learners fit on, `data.spec` the `DataSpec` describing what each of its columns means (which raw attribute, which test). Boolean is only one of three equivalent internal representations `pyrulearn` can use – the others are `SparseDataRepresentation` and `NListRepresentation` (itemset-based).

Every feature's name, in full:

```
[3]: import textwrap

print(textwrap.fill(", ".join(data.spec.feature_names), width=80))
```

pclass=1, pclass=2, pclass=3, pclass!=1, pclass!=2, pclass!=3, sex=female, sex=male, age>=8.5, age>=60.2, age<8.5, age<60.2, sibsp=0, sibsp=1, sibsp=2, sibsp=3, sibsp=4, sibsp=5, sibsp=8, sibsp!=0, sibsp!=1, sibsp!=2, sibsp!=3, sibsp!=4, sibsp!=5, sibsp!=8, parch=0, parch=1, parch=2, parch=3, parch=4, parch=5, parch=6, parch=9, parch!=0, parch!=1, parch!=2, parch!=3, parch!=4, parch!=5, parch!=6, parch!=9, fare>=15.2, fare>=76.0, fare<15.2, fare<76.0, embarked=C, embarked=Q, embarked=S, embarked!=C, embarked!=Q, embarked!=S

A peek at a few rows and columns (the full matrix is much bigger):

```
[4]: import pandas as pd
from IPython.display import display

df_bool = pd.DataFrame(data.X, columns=data.spec.feature_names)
with pd.option_context("display.max_columns", 8, "display.max_rows", 6):
    display(df_bool)
```

	pclass=1	pclass=2	pclass=3	pclass!=1	...	embarked=S	embarked!=C	\
0	True	False	False	False	...	True	True	
1	True	False	False	False	...	True	True	
2	True	False	False	False	...	True	True	
...	...	...	...	...	...	...	...	
1306	False	False	True	True	...	False	False	
1307	False	False	True	True	...	False	False	
1308	False	False	True	True	...	True	True	

	embarked!=Q	embarked!=S
0	True	False
1	True	False
2	True	False
...	...	...
1306	True	True
1307	True	True
1308	True	False

[1309 rows x 52 columns]

3 Fit a rule learner

pyrulearn features several native re-implementations of well-known rule-learning algorithms, as well as interfaces to their original implementations. They all can be called on a uniform data representation, and return different types of rule-based models (rule sets, lists, etc.) in a uniform representation.

Pypper is a version of the classic RIPPER algorithm. Every learner fits the same way: `fit(data)`.

```
[5]: from pyrulearn.learners.seco import Pypper

model = Pypper().fit(data)
```

4 Inspect the rules

Prolog is `to_string()`'s default format; the rules (and the whole model) also render in a few other formats via `to_string(fmt=...)`, useful for different purposes, from reading them as logic to a compact positional encoding. (`ascii=True` sticks to plain ASCII connectives – `to_string()`'s default uses unicode connectives instead.)

```
[6]: # prolog, pretty-printed: easier to scan for rules with many conditions
print(model.to_string(pretty=True))
```

```
survived(X) :- % (233/17)
    pclass(X, V1), V1 \= 3,
    sex(X, female).
survived(X) :- % (73/4)
    sex(X, female),
    parch(X, V1), V1 \= 1,
    parch(X, V2), V2 \= 5,
    fare(X, V3), V3 >= 15.2,
    embarked(X, V4), V4 \= S.
survived(X) :- % (24/1)
    pclass(X, V1), V1 \= 3,
    age(X, V2), V2 < 8.5,
    fare(X, V3), V3 >= 15.2.
survived(X) :- % (11/2)
    pclass(X, 3),
    parch(X, 1),
    fare(X, V1), V1 >= 15.2,
    embarked(X, C).
% default
died(X) :- true. % (809/500)
```

```
[7]: # readable logic: "if ... then ..."
print(model.to_string(fmt="logic", ascii=True))
```

```
if    pclass != 3 AND sex = female -> survived % (233/17)
elif  sex = female AND parch != 1 AND parch != 5 AND fare >= 15.2 AND embarked !=
```

```
S -> survived % (73/4)
elif pclass != 3 AND age < 8.5 AND fare >= 15.2 -> survived % (24/1)
elif pclass = 3 AND parch = 1 AND fare >= 15.2 AND embarked = C -> survived %
(11/2)
else -> died % (809/500)
```

```
[8]: # bare conditions, no connective or head
print(model.to_string(fmt="conditions", ascii=True))
```

```
survived: pclass != 3, sex = female % (233/17)
survived: sex = female, parch != 1, parch != 5, fare >= 15.2, embarked != S %
(73/4)
survived: pclass != 3, age < 8.5, fare >= 15.2 % (24/1)
survived: pclass = 3, parch = 1, fare >= 15.2, embarked = C % (11/2)
died: % (809/500 - default)
```

```
[9]: # positional 1/0 pattern, one token per feature, 1 where the rule has
# that condition -- pretty=True shows the full vector (otherwise, with
# many features, it truncates)
print(model.to_string(fmt="pattern", pretty=True))
```

```
survived: 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
survived: 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
1 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 1 % (73/4)
survived: 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 % (24/1)
survived: 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 % (11/2)
died: % (809/500 - default)
```

Small aggregate stats – rule count, length, coverage:

```
[10]: from pyrulearn.evaluation import summarize

summarize(model, data)
```

```
[10]: {'n_rules': 4,
      'mean_length': 3.5,
      'median_length': 3.5,
      'rules_per_target': {'survived': 4},
      'mean_n_covered': 91.25}
```

5 Compare two learners across a few datasets

pyrulearn can access about a hundred small, medium, and large benchmark datasets through its Catalog, fetched automatically through OpenML (100 of its 104 entries; the rest are pointer-only, to datasets found elsewhere).

```
[11]: from pyrulearn.experiments.catalog import Catalog
```

```
catalog = Catalog.default()
print(len(catalog.entries), "datasets, e.g.:")
for e in catalog.entries[:5]:
    print(f"{e.name:28s} {e.task:10s} {e.size:6s} {e.n_instances:>6} rows")
```

104 datasets, e.g.:

adult	binary	large	48842 rows
analcatauthorship	multiclass	small	841 rows
analcatauthorship_dmft	multiclass	small	797 rows
anneal	multiclass	small	898 rows
audiology	multiclass	small	226 rows

`run_cv` handles the cross-validation, binarization per fold, timing and failure-handling for you, discretizing each fold the same way `read_csv` did above, via its own `max_intervals=`. Here's Pypper against PFossil (another native separate-and-conquer learner) on three small datasets:

```
[12]: from pyrulearn.experiments.runner import run_cv
from pyrulearn.learners.seco import PFossil

datasets = catalog.select(names=["vote", "tic-tac-toe", "hepatitis"])
results = run_cv([Pypper(), PFossil()], datasets, n_folds=5, max_intervals=8,
    verbose=False)
results.groupby(["dataset", "learner"])[["accuracy", "n_rules", "fit_time"]].
    mean().round(3)
```

```
[12]:
```

		accuracy	n_rules	fit_time
dataset	learner			
hepatitis	PFossil	0.839	14.0	0.073
	Pypper	0.755	1.2	0.031
tic-tac-toe	PFossil	0.974	14.8	0.071
	Pypper	0.983	8.0	0.155
vote	PFossil	0.956	8.6	0.019
	Pypper	0.961	2.4	0.277

Summary by learner – mean across datasets and folds, plus mean rank (1 = best) – the same summary every comparison demo in `demos/` reports:

```
[13]: from pyrulearn.experiments.stats import mean_rank

summary = results.groupby("learner")[["accuracy", "n_rules", "fit_time"]].mean()
summary["mean_rank"] = mean_rank(results, "accuracy")
summary.round(3)
```

```
[13]:
```

	accuracy	n_rules	fit_time	mean_rank
learner				
PFossil	0.923	12.467	0.054	1.667
Pypper	0.900	3.867	0.155	1.333

6 Where to go from here

- `README.md` – a detailed description of the package, with a summary of all provided functionalities, including a list of all native and interfaced learners.
- `demos/` – demos of different functionalities. Each can be called in a quick mode (default), which demonstrates its base functionality, and a full mode, which does a more elaborate investigation of an issue. Sample outputs are enclosed in the checked-in reports.